



- Objectifs du projet
- Contraintes du sujet
- Philosophie attendue
- Pourquoi RL
- Boucle RL
- Concepts clés
- Exploration vs exploitation
- Q-learning
- SARSA
- Autres algorithmes
- Environnement Taxi
- Méthodologie recommandée
- Protocol expérimental
- Hyperparamètres
- Analyse
- Extensions avancées
- Ce qui est attendu

Rappels/conseils

Objectif du projet

Concevoir un agent capable de résoudre Taxi-v3 via apprentissage par renforcement.

L'agent doit apprendre à :

- naviguer dans un environnement discret
- récupérer un passager
- atteindre la destination correcte
- minimiser le nombre d'étapes



- optimisation $\neq$ juste réussir
- votre agent doit apprendre une stratégie optimale sous contrainte de coût
- reward shaping = boussole

Contraintes du sujet



Contraintes du projet :

- ≥ 2 algorithmes (baseline + avancé)
- Mode tuning + mode optimal
- Benchmark complet (metrics + graphes)
- Analyse des résultats



- Très important : benchmark obligatoire
- GUI secondaire, focus IA

Livrables :

- Code
- Visualisations
- Rapport synthétique

Philosophie attendue

Approche attendue

Projet = démarche scientifique :

- formuler des hypothèses
- expérimenter
- comparer
- analyser
- justifier



- Pas de “ça marche donc c’est bon”
- On veut comprendre les comportements
- Exemple hypothèse : “ γ élevé $\rightarrow$ convergence plus lente mais meilleure politique”



Pourquoi RL

Pourquoi Reinforcement Learning ?

Apprentissage par interaction avec un environnement.

Cas d'usage :

- robotique
- jeux (jeux vidéo, AlphaGo)
- systèmes autonomes
- optimisation séquentielle



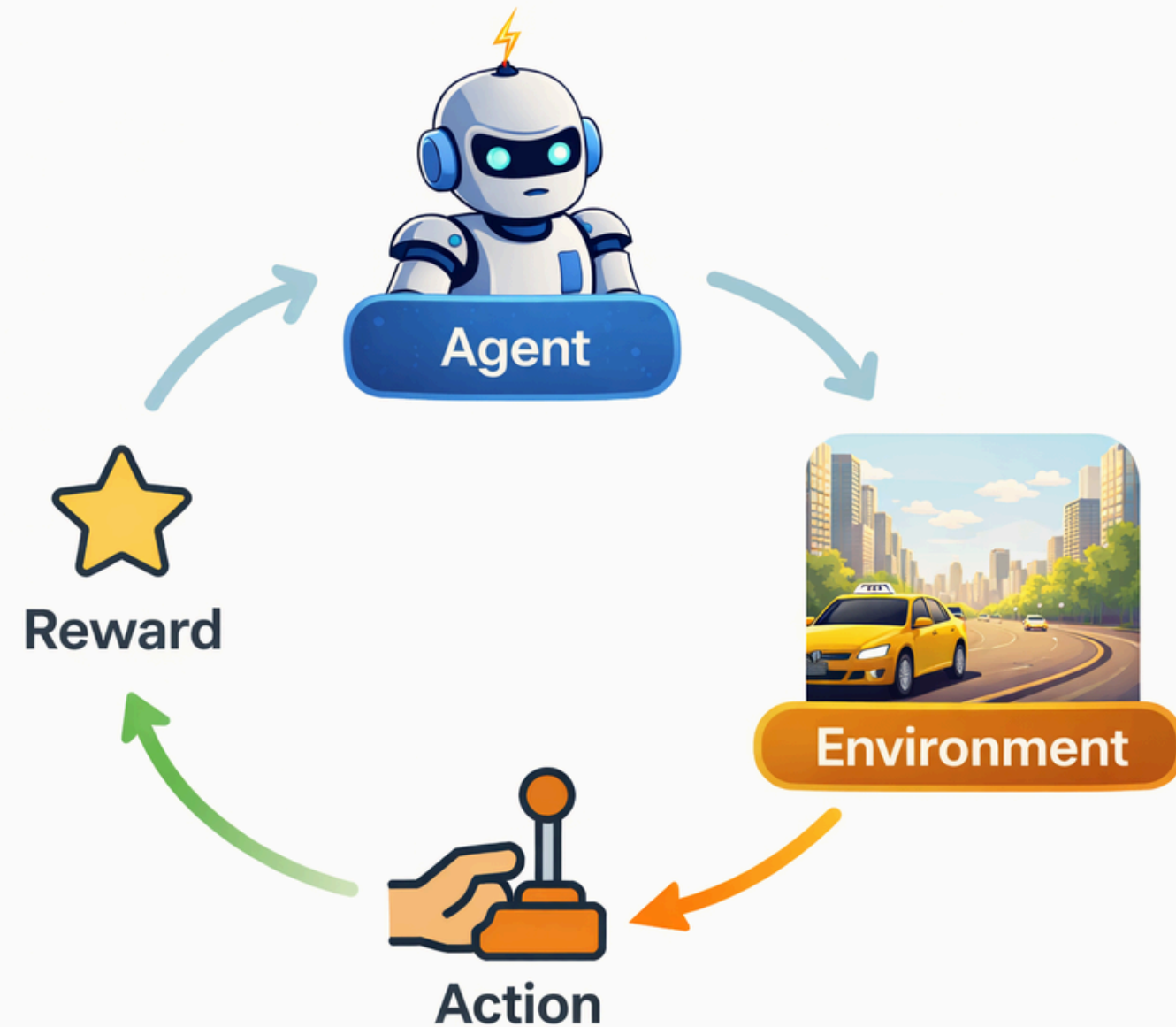
- Décision séquentielle $\neq$ classification
- Une bonne décision locale peut être globalement mauvaise (et inversement)

Boucle RL

Boucle d'apprentissage

1. observer état **S**
2. choisir action **A**
3. recevoir récompense **R**
4. observer **S'**
5. mettre à jour Q ou la politique (selon l'algo)

Objectif : maximiser la récompense cumulée



- Bien comprendre le boucle de feedback

Concepts fondamentaux

- **État (S)**

Représentation de la situation actuelle de l'environnement

→ ex : position du taxi, du passager, destination

- **Action (A)**

Décision prise par l'agent

→ ex : déplacer, prendre, déposer

- **Récompense (R)**

Signal de feedback immédiat

→ guide l'apprentissage (positif ou négatif)

- **Politique (π)**

Stratégie de décision

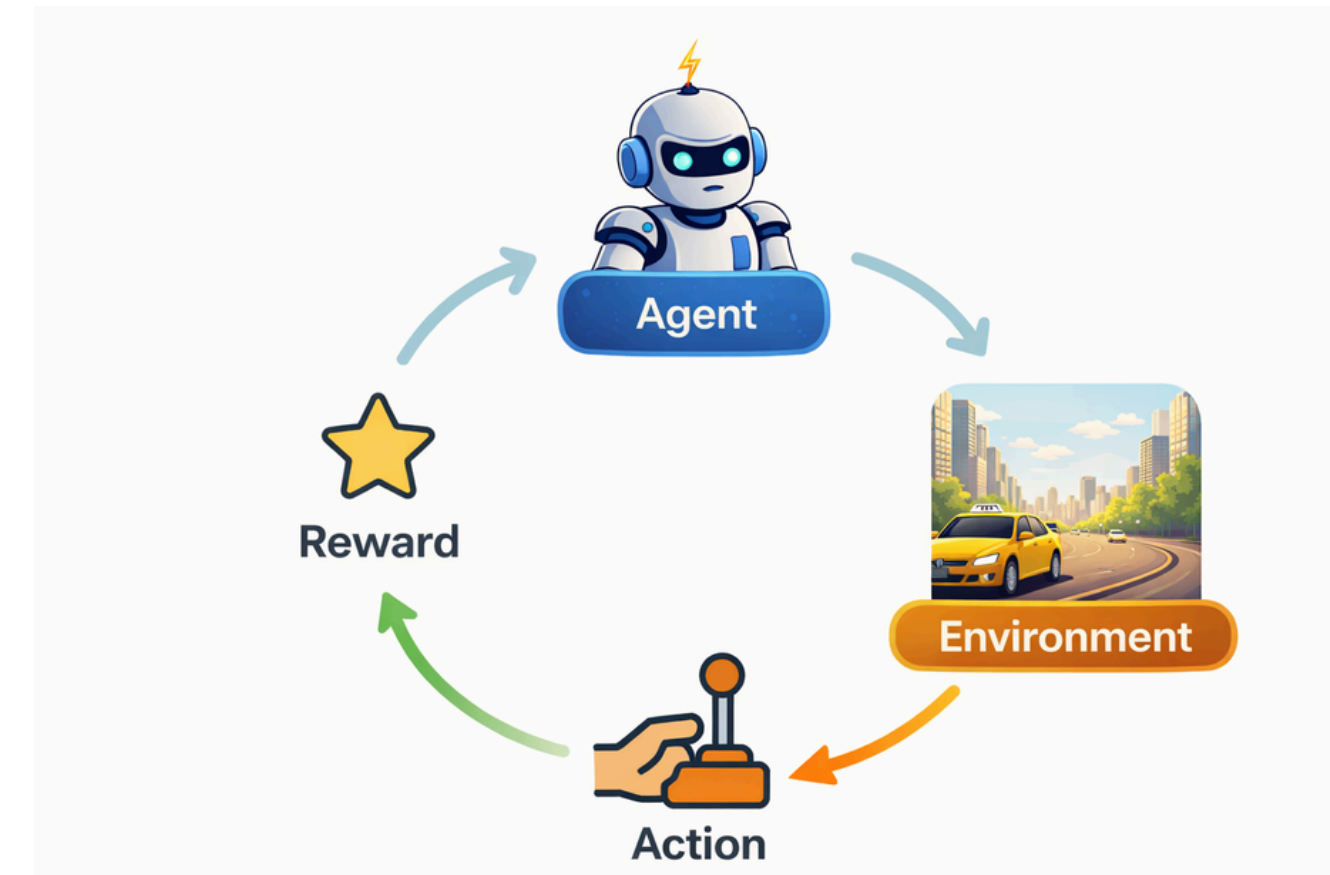
→ $\pi(s)$ = action choisie dans l'état s

- **Valeur $Q(s,a)$**

Qualité d'une action dans un état

→ espérance de récompense future

Concepts clés



- la modélisation définit le problème
- $Q(s,a)$ = combien ça vaut de faire ça ici, à long terme.

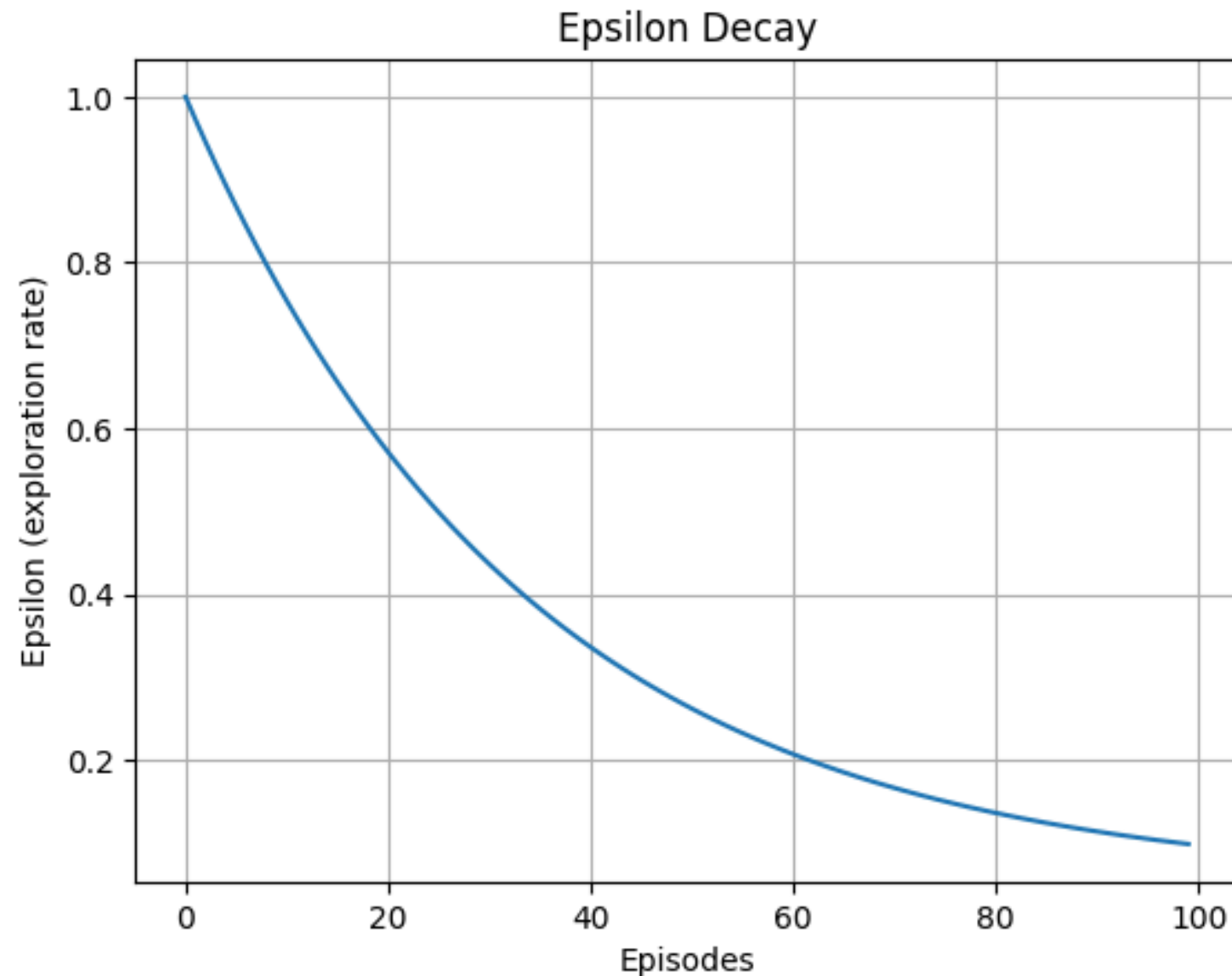
Exploration vs Exploitation

- Exploration : découvrir
- Exploitation : optimiser

Trade-off:

- trop explorer → lent
- trop exploiter → sous-optimal

Méthode : ϵ -greedy



- Epsilon Decay $\neq$ Learning rate decay
- Sans exploration, l'agent apprend une mauvaise stratégie très vite.

Q-Learning

Q-Learning (off-policy)

Apprend $Q(s,a)$ = espérance de gain

Update :

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Paramètres :

- α : learning rate
- γ : discount factor

État S	Sud	Nord	Est	Ouest	Pickup	Dropoff	Meilleure action
s_0	-1.2	-0.8	0.4	-0.6	-10.0	-10.0	Est
s_1	0.7	-0.4	1.3	0.2	-10.0	-10.0	Est
s_2	-0.9	0.1	-0.3	1.8	-10.0	-10.0	Ouest
s_3	-0.5	-0.2	0.6	0.4	4.5	-10.0	Pickup
s_4	-0.7	0.3	-0.1	0.2	-10.0	8.0	Dropoff



- off-policy → regarde le meilleur futur
- Q-learning suppose que dans le futur l'agent jouera parfaitement

SARSA

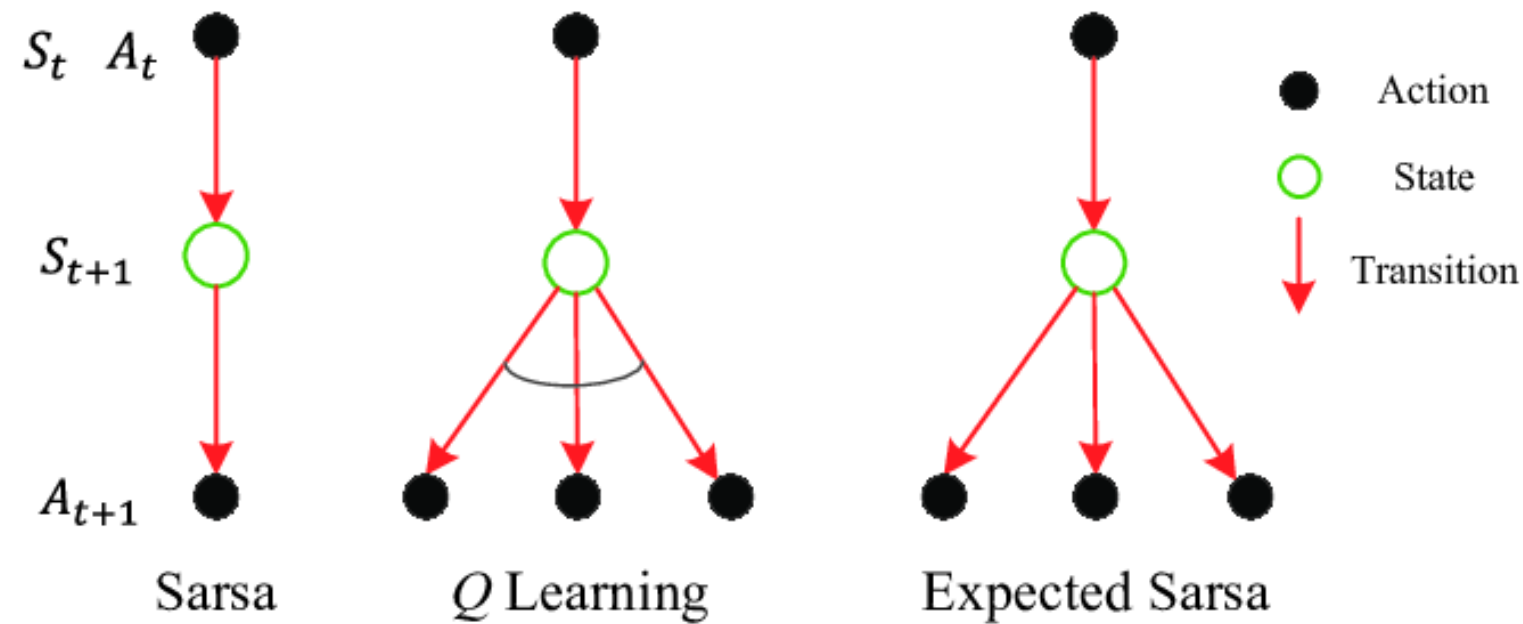
SARSA (on-policy)

Update :

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

Caractéristiques :

- on-policy
- plus conservateur



- dépend de la policy réelle
- SARSA apprend avec ses erreurs, Q-learning apprend comme s'il n'en faisait pas.

Q-learning vs SARSA



SARSA

nouvelle valeur =
ancienne valeur
+ learning_rate × (
reward
+ gamma × **valeur de l'action réellement choisie ensuite**
- ancienne valeur
)

- *plus stable*
- *plus robuste à l'exploration*
- *mais parfois moins performant*

Q-learning

nouvelle valeur =
ancienne valeur
+ learning_rate × (
reward
+ gamma × **meilleure valeur possible dans l'état suivant**
- ancienne valeur
)

- *converge plus vite*
- *trouve souvent une meilleure solution*
- *mais peut être instable si mal tuné*

(Quelques) Autres algorithmes

Monte Carlo	~1950s	Apprend uniquement à la fin des épisodes (pas de mise à jour intermédiaire)
SARSA	1988	On-policy → apprend avec les actions réellement prises
Q-learning	1989	Off-policy → apprend avec la meilleure action possible (optimiste)
Expected SARSA	~2009	Version plus stable : moyenne des actions au lieu d'un seul choix
Dyna-Q	1990	Combine apprentissage + simulation (model-based + model-free)
REINFORCE	1992	Apprend directement la politique (pas de Q-table)
DQN	2013	Q-learning + réseau de neurones (passage au deep learning)
A2C / A3C	~2016	Actor-Critic stable avec apprentissage parallèle
DDPG	2015	Actions continues (Q-learning + policy gradient)
PPO	2017	Policy gradient stabilisé (standard industriel)
SAC	2018	Ajoute exploration via entropie → très robuste
TD3	2018	Corrige les biais de DDPG (plus stable)
MCTS	~2006	Planification par simulation (utilisé dans AlphaGo)
CQL	~2020	Offline RL → apprend sans interaction



Environnement Taxi

Taxi-v3

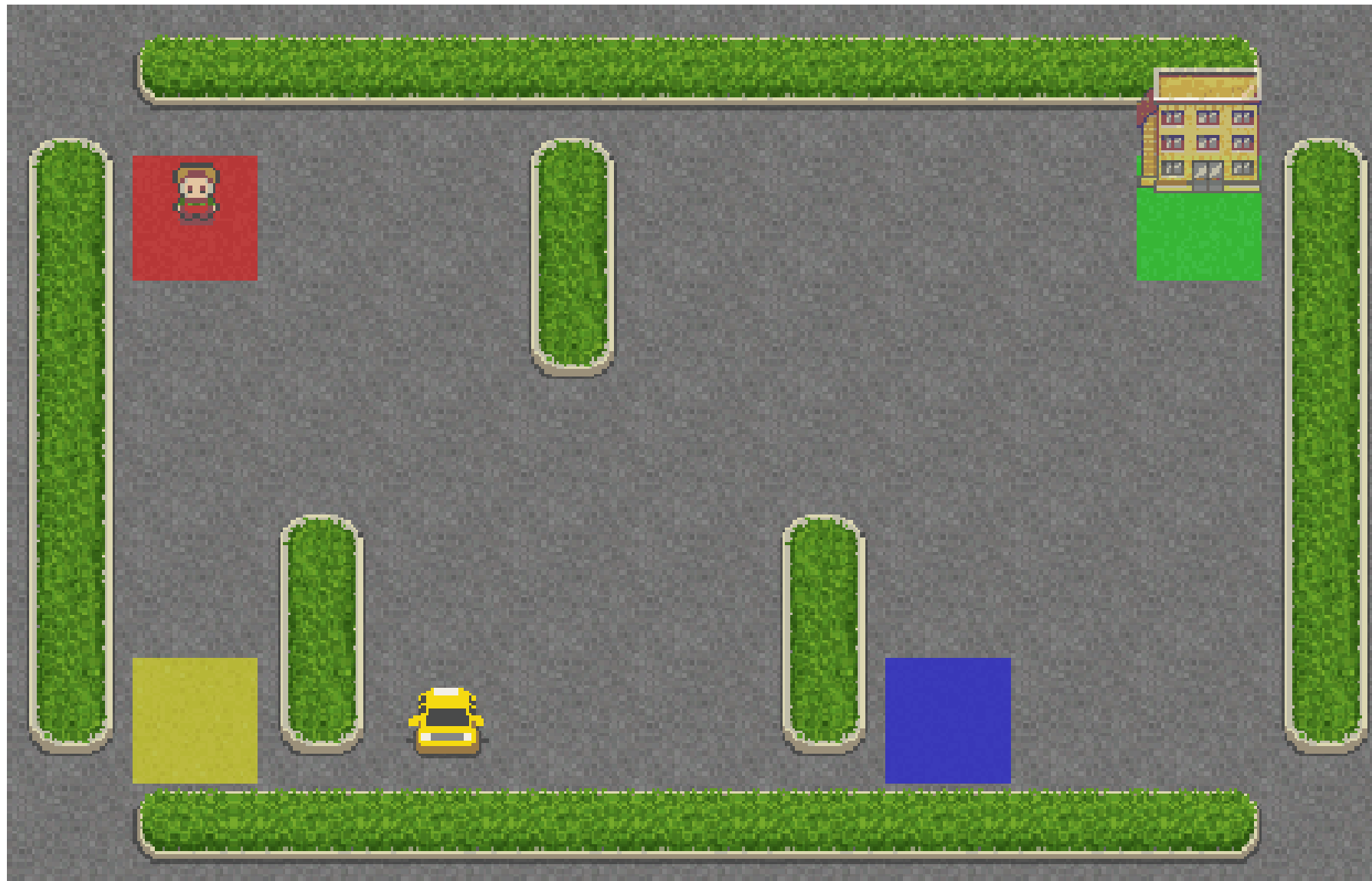
- environnement discret
- 500 états
- 6 actions

Problème :

- navigation + logique (pickup/drop)

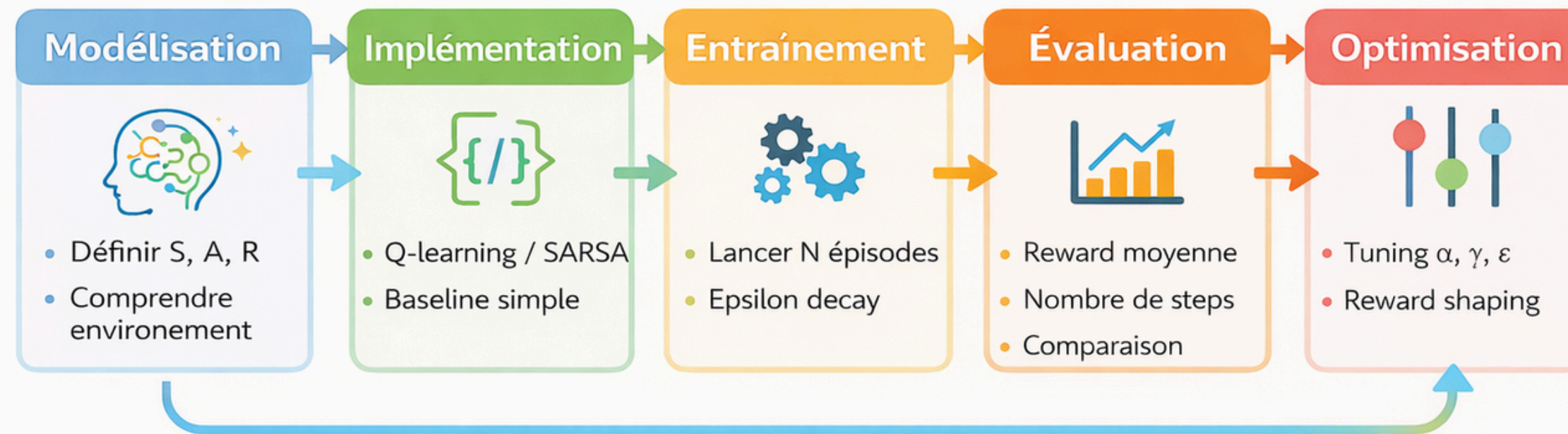


- parfait pour tabulaire
- c'est simple en apparence, mais combinatoire non triviale



Méthodologie recommandée

Pipeline de développement



RL = itération expérimentale + métriques + amélioration

1. agent “brute force” (baseline)
2. Q-learning
3. SARSA
4. tuning hyperparamètres
5. extension (Deep RL ?)



- incrémental

Protocole expérimental

Éléments de comparaison rigoureuse

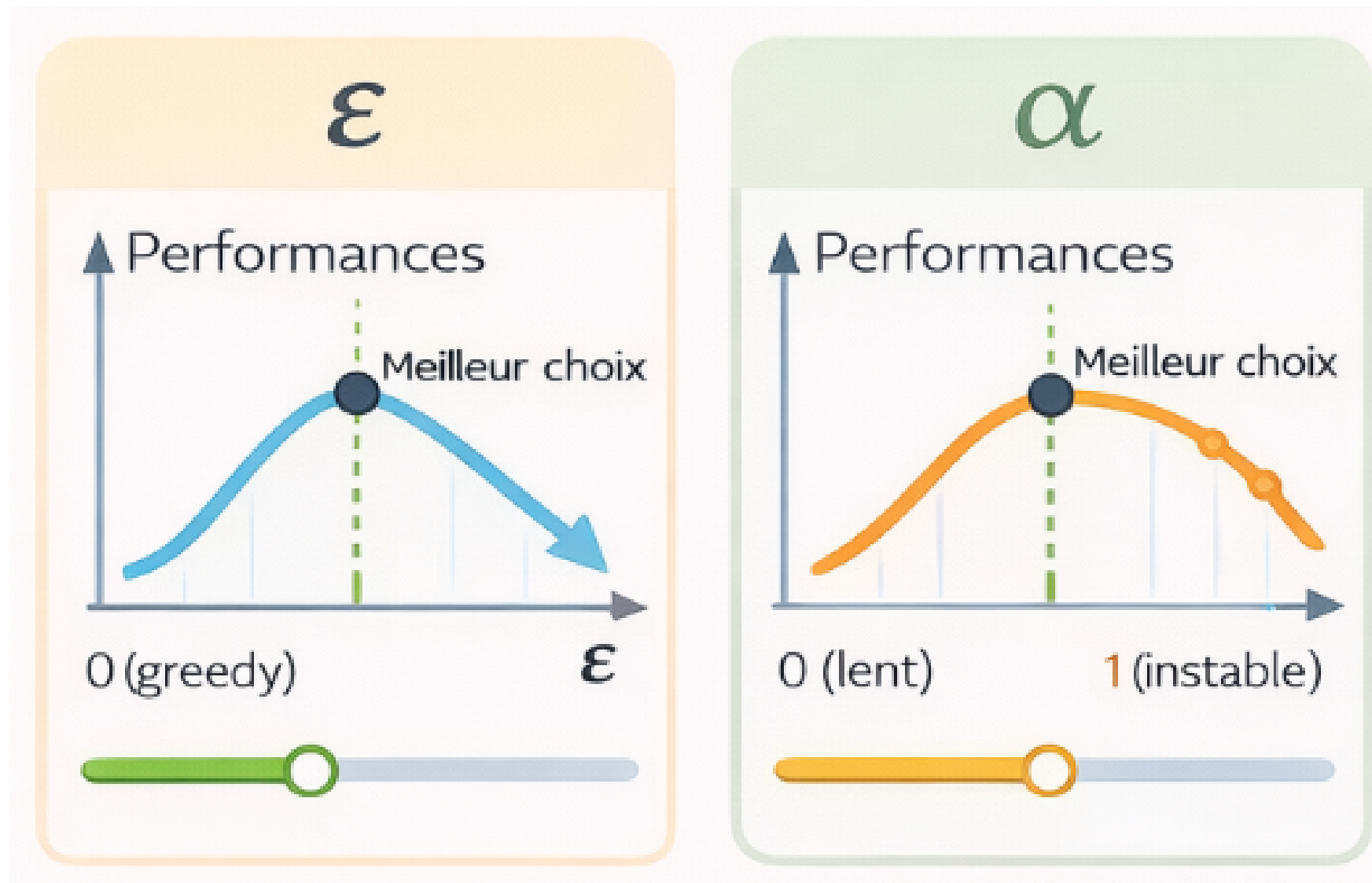
Mesures :

- reward moyen
- steps moyens
- taux de succès
- ...



- Il faut que les résultats soient comparables
- notion de variance / répétitions

Hyperparamètres



Paramètres critiques

- α (learning rate)
- γ (discount)
- ϵ (exploration)

Expérimentations :

- grid search simple
- epsilon decay



- impact majeur sur convergence
- les résultats peuvent être plus sensibles aux hyperparamètres qu'à l'algorithme

Analyse



Analyse

Analyse des résultats

J'insiste là dessus :

- Il faut que le rapport raconte une histoire
- pas juste un empilement d'expériences

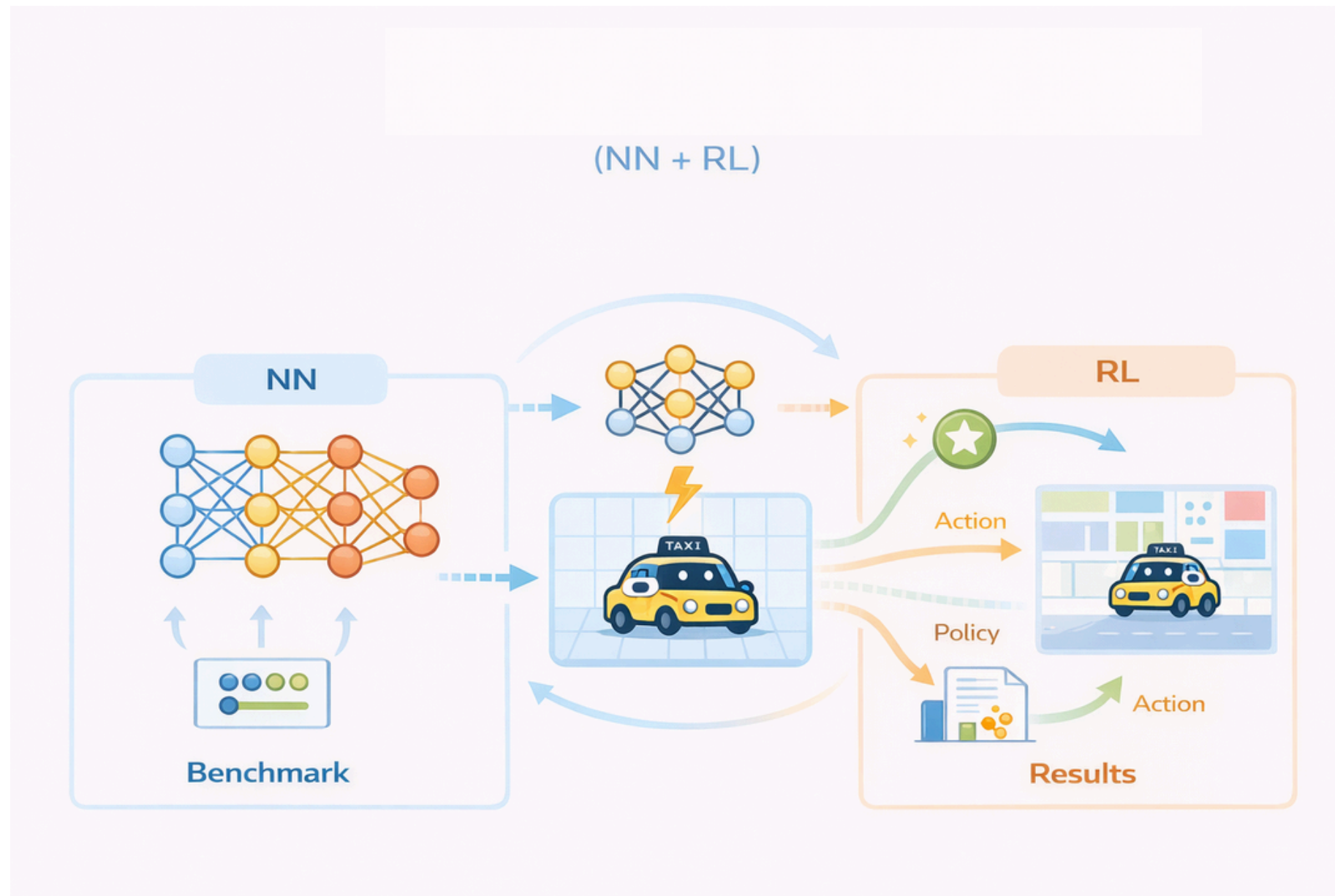


- méthodes scientifique
- un bon projet IA, c'est un bon rapport, pas juste un bon code

Extensions avancées

Aller plus loin

- Deep Q-Learning
- approximation par réseaux de neurones
- environnements plus complexes
- modification du problème



- Q-table → limite de passage à l'échelle
- DQN, c'est juste Q-learning + réseau de neurones

Ce qui est attendu

Projet solide

- ≥ 2 algorithmes (ce serait bien d'en avoir plus 😊)
- benchmark propre
- visualisations
- analyse critique
- limites + améliorations
- travail de groupe



- compréhension > performance brute

